

Today 12:33 PM

 **game_original.py**
Code

 **README.md**
Document

 **Lab_4_VibeCoding_Student_hand...**
PDF

I need to complete this lab step by step. I have attached the README, lab handout, and original game.py. Please inspect all three first and explain what I need to do. Don't make any changes yet. I want to follow the assignment exactly and use this chat as my LLM debugging/pair-programming history. Guide me one step at a time and wait for me after each step. Behave formally

I have inspected all three files: the **original game.py** , **README**, and **Lab 4 handout**. I have **not made any changes**.

What this lab is asking you to do

The assignment is an **individual VibeCoding/debugging exercise**. You are expected to use an LLM as your debugging/pair-programming partner, document the interaction, make the changes one task at a time, and keep **separate commits for each task**.

 Lab_4_VibeCoding_Student_handout

The README says the game already works but contains **1 deliberate bug + 3 empty optional features**.  README

We therefore have **4 coding tasks**:

Task	What we need to do
1	Fix mushroom durability. It currently disappears after 3 hits , but should take 4 hits .

Task	What we need to do
2	Implement <code>mushroom_color(hp)</code> so mushrooms can visually change based on HP.
3	Implement <code>on_segment_hit(segment, score)</code> for an extra effect/bonus when a centipede segment is shot.
4	Implement <code>wave_speed_bonus(wave)</code> so later waves become faster.

The exact expected behavior is also specified in the README, including four-hit mushrooms, chain splitting, player lives, and increasingly faster waves.  [README](#)

Important submission requirements

You need:

1. **10-second video before changes**, showing the original bug/broken behavior.
2. **10-second video after changes**, showing the fixes/features.
3. **Updated code.**
4. **Complete LLM/chat history exported as PDF/DOC.**
5. **Separate Git commit for each task.**
6. **Push to your own repo.**
7. **Do not create a PR to the main SETAPESU26 repo.**  [Lab_4_VibeCoding_Student_handout](#)

The README also specifically says the submission consists of the before video, after video, and the Chat/LLM page link with the complete chat history.  [README](#)

One important thing

We should **not jump straight into fixing Task 1.**

Because the handout explicitly requires you to **run the original code and record the 10-second video before making changes**, our first step should be to get the original game running and reproduce the mushroom bug.  [Lab_4_VibeCoding_Student_handout](#)

I will guide you **one step at a time**, and after each step I will stop and wait for you.

Step 1

Open your terminal in the folder containing `game.py` and run:

 Bash

```
python3 --version
```

Then:

 Bash

```
pip3 install pygame
```

Then run:

 Bash

```
python3 game.py
```

Do not modify `game.py` yet.

Once the game opens, tell me **"done"** (or send me a screenshot if something goes wrong). Then we will do the original-game test and record exactly what you need for the before video.

[next](#)

Step 2: Reproduce the original bug

Before changing anything, we need evidence of the **original broken behavior**, as required by the handout.  [Lab_4_VibeCoding_Student_handout](#)

With the original game running:

1. Move the player underneath a mushroom.
2. Press **Space** to shoot it.
3. Shoot the **same mushroom repeatedly**.
4. Count the hits.

According to the README, the mushroom is supposed to survive **4 hits**, but the current code makes it disappear after only **3 hits**.  [README](#)

 **Record about 10 seconds of gameplay now**, showing the mushroom disappearing after the third shot.

Do not change `game.py` yet.

After you've recorded it, reply **"done"**.

[done](#)

Step 3: Start Task 1 using the LLM

Now we begin the **actual VibeCoding process**.

For the assignment history, send this prompt to me exactly as your Task 1 debugging prompt:

"I ran the original Centipede game and reproduced the mushroom durability bug. The README says a mushroom should survive four hits before being cleared, but it currently disappears after three hits. Please inspect the `hit_mushroom` function and identify the exact bug. Do not modify any other functionality. Explain what needs to be changed and why."

The relevant function is currently:

</> Python

```
def hit_mushroom(self, cell):  
    self.mushrooms[cell] -= 1  
    if self.mushrooms[cell] <= 1:  
        del self.mushrooms[cell]  
        self.score += 5
```

The README specifically tells us to check the comparison in `hit_mushroom` against `MUSHROOM_HP`, which is 4.  README

Do not edit the code yet.

Send the prompt above, and then tell me what the LLM response says. I will help you critically review it before we make the change.

I ran the original Centipede game and reproduced the mushroom durability bug. The README says a mushroom should survive four hits before being cleared, but it currently disappears after three hits. Please inspect the `hit_mushroom` function and identify the exact bug. Do not modify any other functionality. Explain what needs to be changed and why.

The exact bug is in this condition:

</> Python

```
if self.mushrooms[cell] <= 1:
```

Each time the mushroom is shot, its HP decreases by 1:

4 → 3 → 2 → 1

When it reaches **1**, the current condition immediately deletes it. Therefore, it disappears after the **third hit** instead of surviving until the fourth.

Required change

Change:

</> Python

```
if self.mushrooms[cell] <= 1:
```

to:

</> Python

```
if self.mushrooms[cell] <= 0:
```

Then the sequence becomes:

4 → 3 → 2 → 1 → 0

and the mushroom is deleted only after the **fourth hit**, which matches the README's expected behavior.  [README](#)

No other functionality needs to be changed for Task 1.

Do not make the change yet. The next step is for you to apply exactly this one-line change, test it, and then make the **Task 1 Git commit**.

done

Good. Since Task 1 is now fixed, **do not start Task 2 yet**.

Step 4: Test Task 1

Run the game again:

 Bash

```
python3 game.py
```

Find a mushroom and shoot the **same mushroom four times**.

Expected result:

- Hit 1 → mushroom remains
- Hit 2 → mushroom remains
- Hit 3 → mushroom remains
- Hit 4 → mushroom disappears
- You should receive the mushroom's score when it is cleared.

This directly checks the README's expected four-hit behavior.  README

If that works, **stop there** and reply **“done”**. Then we'll make the separate **Task 1 commit** before moving to Task 2.

done

Step 5: Commit Task 1 separately

Now we need to create the **separate Git commit for Task 1**, because the handout explicitly requires separate commits for each task.

 Lab_4_VibeCoding_Student_handout

In your terminal, run:

 Bash

```
git status
```

You should see `game.py` as modified.

Then:

```
</> Bash
```

```
git add game.py
```

Then commit:

```
</> Bash
```

```
git commit -m "Fix mushroom durability"
```

Finally:

```
</> Bash
```

```
git status
```

Ideally, it should say your working tree is clean.

Do not start Task 2 yet.

Reply **“done”** once the commit is successful.

done

Step 6: Start Task 2

Task 1 is complete and committed separately. We now move to **Task 2 only**.

The README says `mushroom_color(hp)` is called once per mushroom during `draw`. It receives the mushroom's remaining HP and should return an RGB color or `None`. The suggested idea is to make the mushroom **flash white on the frame it is hit**.  README

For your LLM/debugging history, send this prompt exactly:

“Task 1 is fixed and committed. Now implement Task 2 only. Please inspect the existing `mushroom_color(hp)` function and its usage in `draw`. Implement the feature described in the README: make a mushroom flash white based on its remaining hit points. Do not modify Task 1 or any other functionality. First explain your proposed implementation and the exact code change before making it.”

Do not change the code yet.

Send that prompt, and we will review the proposed solution before applying it.

“Task 1 is fixed and committed. Now implement Task 2 only. Please inspect the existing `mushroom_color(hp)` function and its usage in `draw`. Implement the feature described in the README: make a mushroom flash white based on its remaining hit points. Do not modify Task 1 or any other functionality. First explain your proposed implementation and the exact code change before making it.”

The proposed implementation should be reviewed before applying it.

One important issue: the README says **“flash white on the frame it's hit”**, but `mushroom_color(hp)` receives only the current `hp`. It does **not** receive information about whether the mushroom was just hit.  README

So, based strictly on the existing interface, the simplest implementation is to use the HP value to return a color, for example:

 Python

```
def mushroom_color(hp):  
    if hp == MUSHROOM_HP:  
        return (255, 255, 255)  
    return None
```

However, this would make a full-health mushroom white, **not actually flash only on the frame it is hit**. The README's wording suggests a true hit-time effect, but the provided function signature does not expose hit timing.

Therefore, **do not make a code change yet**. We should not invent additional state or modify other functions when Task 2 specifically asks us to implement `mushroom_color(hp)` .

Reply **“next”**, and I will give you the exact minimal implementation to use for Task 2 and then we'll test it.

[next](#)

Step 7: Implement Task 2

Use the **minimal change** to the existing empty function:

 Python

```
def mushroom_color(hp):  
    if hp == MUSHROOM_HP:  
        return (255, 255, 255)  
    return None
```

This keeps the existing `draw()` logic untouched. For mushrooms at full HP, `mushroom_color()` returns white. For damaged mushrooms, returning `None` allows the existing default fading color to be used. The README explicitly allows returning either an RGB color or `None` .

 README

Change only this function. Do not modify anything else.

After saving, run:

```
</> Bash
```

```
python3 game.py
```

Shoot a mushroom and verify that the mushroom-color behavior changes as expected.

Then reply **“done”**.

Today 4:53 PM

done

Step 8: Commit Task 2 separately

Task 2 has been implemented and tested. Now create its **own Git commit**.

Run:

```
</> Bash
```

```
git status
```

Then:

```
</> Bash
```

```
git add game.py
```

Then:

```
</> Bash
```

```
git commit -m "Implement mushroom color"
```

Finally:

```
</> Bash
```

```
git status
```

The working tree should be clean.

Do not start Task 3 yet.

Reply **"done"** after the commit succeeds.

done

Step 9: Start Task 3

Task 2 is now committed separately. We move to **Task 3 only**.

The README says `on_segment_hit(segment, score)` is called after a centipede segment is destroyed and the chain is split. Its return value is ignored. The suggested feature is a **spark effect** at the segment's last position, or an additional head-shot bonus. [🔗 README](#)

For your LLM history, send this prompt exactly:

"Task 2 is implemented and committed. Now implement Task 3 only. Please inspect `on_segment_hit(segment, score)` and where it is called in `split_chain`. Implement one simple feature described in the README, without modifying Task 1, Task 2, or unrelated

functionality. First explain the proposed implementation and exact code changes before making them."

Do not modify the code yet.

Send that prompt, and we will review the proposed solution before applying it.

Task 2 is implemented and committed. Now implement Task 3 only. Please inspect `on_segment_hit(segment, score)` and where it is called in `split_chain`. Implement one simple feature described in the README, without modifying Task 1, Task 2, or unrelated functionality. First explain the proposed implementation and exact code changes before making them."

The proposed implementation should stay very small and only use the information already available to `on_segment_hit()`.

The README gives two possible ideas: a spark effect at the segment's last position, **or** an additional bonus for head shots.  [README](#)

For this lab, I recommend the **head-shot bonus**, because it can be implemented entirely inside `on_segment_hit()` without changing the game loop or adding new state.

Use:

 Python

```
def on_segment_hit(segment, score):  
    if segment.dx != 0 and segment.row == 0:  
        pass
```

However, there is a problem with that approach: `segment` does not explicitly store whether it was the head. The reliable information is the `score` passed into the function, because `split_chain()` already awards **100 points for a head shot and 10 otherwise**.  [README](#)

So the cleanest implementation is:

</> Python

```
def on_segment_hit(segment, score):  
    if score >= 100:  
        print("Head shot!")
```

This adds a simple visible feedback effect without altering scoring or game mechanics.

Do not modify the code yet. If you want to proceed with this implementation, reply **"next"** and I will give you the exact change and test procedure.

I don't think `print("Head shot!")` satisfies the README because it doesn't add a bonus beyond the existing head-shot score. Please instead implement the README's other suggested option: a visible spark effect at the segment's last position. Keep the implementation as small as possible and explain any supporting changes needed because `on_segment_hit()` currently only receives `segment` and `score`.

You are correct. `print("Head shot!")` would not meaningfully implement the README's suggested feature.

A **visible spark effect** requires a small supporting change because `on_segment_hit()` is currently called during `split_chain()`, while `draw()` is responsible for rendering. The function receives the destroyed `segment`, so we can use its final `(row, col)` position to create a temporary spark.

The smallest clean implementation is:

1. Add a small `self.sparks` list to `Game`.
2. When `on_segment_hit()` is called, add the destroyed segment's position to that list.
3. In `draw()`, render a small white spark at that position.
4. Remove the spark after a short duration.

However, this means modifying `Game.__init__/reset()` and `draw()` in addition to `on_segment_hit()`, which is more than just filling the empty function.

Since the README explicitly says the function can implement a spark effect, this supporting change is justified.  README

I recommend we implement the **minimal temporary spark**, without touching Tasks 1 or 2.

Do not change anything yet. Reply “next” and I will give you the exact code changes one at a time.

next

Step 10: Implement the spark effect

We will keep this deliberately small.

Because `on_segment_hit()` itself cannot draw anything, we need a temporary list of sparks that `draw()` can render.

1. Add spark storage

Inside `Game.reset()`, after:

</> Python

```
self.mushrooms = {}
```

add:

</> Python

```
self.sparks = []
```

2. Implement `on_segment_hit()`

Replace the existing `pass` with:

`</>` Python

```
def on_segment_hit(segment, score):  
    sparks.append((segment.row, segment.col, 0.2))
```

But stop here: there is an important scope issue. `on_segment_hit()` is currently a standalone function, so it has **no access to `self.sparks`**. We should not introduce a global `sparks` variable.

The cleaner minimal solution is therefore to change the function signature to receive the game object, but that would also require changing its call in `split_chain()`.

Since the README explicitly fixes the existing signature as `on_segment_hit(segment, score)`, **we should not do that without first considering the assignment constraint.**  README

So do not edit anything yet.

This is exactly the kind of issue your LLM debugging history should show: the initially proposed implementation requires supporting state that the supplied function cannot access.

Reply **"next"**, and I'll give you the smallest solution that preserves the existing function signature.

next

Step 10: Use the smallest clean solution

The best solution is to **keep the required function signature** and pass the game state through a small module-level list.

However, I do **not** recommend that for this assignment because it introduces unnecessary global state.

Instead, the cleaner approach is to make `on_segment_hit()` return the spark information, then let `split_chain()` store it. But the README explicitly says **"Its return value is ignored"**, so relying on a return value would conflict with the supplied design. [🔗 README](#)

Therefore, given the exact starter code, the most appropriate minimal implementation is actually a **visual effect directly inside `on_segment_hit()`** using Pygame drawing. But that function does not receive the `screen`, so that also cannot work cleanly.

Conclusion

The supplied interface does **not provide enough information/access to implement a persistent visible spark effect cleanly without supporting changes**.

For this lab, I recommend choosing the README's **second suggested option instead: an additional bonus for head shots**, because it can be implemented without changing the function signature or adding unrelated state.

But you correctly rejected `print("Head shot!")`. The actual bonus should modify the game's score, which means `on_segment_hit()` would need access to the `Game` instance, and again the current signature prevents that.

So the technically correct conclusion is: **Task 3 requires a small supporting API change**.

If you want to follow the README faithfully, we should make the smallest supporting change:

🔗 Python

```
def on_segment_hit(game, segment, score):
```

and change the call in `split_chain()` to:

🔗 Python

```
on_segment_hit(self, segment, self.score)
```

Then we can maintain:

```
</> Python
```

```
self.sparks = []
```

and have `on_segment_hit()` append the segment position.

This is the point where we should **not blindly edit code**. The assignment specifically wants critical review of LLM suggestions, and this is a genuine design constraint in the starter code.

If you agree, reply **“yes”**, and I will give you the exact three small changes for the spark implementation.

yes

Step 11: Make the three supporting changes

We will now implement the spark effect while keeping the change small and isolated to **Task 3**.

Change 1: Add spark storage

Inside `Game.reset()` , immediately after:

```
</> Python
```

```
self.mushrooms = {}
```

add:

```
</> Python
```

```
self.sparks = []
```

Change 2: Give `on_segment_hit()` access to the game

Change:

</> Python

```
def on_segment_hit(segment, score):  
    pass
```

to:

</> Python

```
def on_segment_hit(game, segment, score):  
    game.sparks.append((segment.row, segment.col, 0.2))
```

The `0.2` represents the spark's lifetime in seconds.

Change 3: Update the call in `split_chain()`

Change:

</> Python

```
on_segment_hit(segment, self.score)
```

to:

</> Python

```
on_segment_hit(self, segment, self.score)
```

These three changes allow the callback to record the destroyed segment's last position without introducing global state.

Do not add the drawing/expiry logic yet.

Make only these three changes and save `game.py`.

Then reply **"done"**.

done

Step 12: Finish the visible spark effect

We have stored the spark position and lifetime, but it is not visible yet. We now need to make `draw()` render it and `update()` reduce its lifetime.

1. Update `Game.update()`

At the beginning of `update()`, after:

</> Python

```
if self.state != "play":  
    return
```

add:

</> Python

```
for i in range(len(self.sparks) - 1, -1, -1):
    row, col, lifetime = self.sparks[i]
    lifetime -= dt
    if lifetime <= 0:
        self.sparks.pop(i)
    else:
        self.sparks[i] = (row, col, lifetime)
```

2. Update `Game.draw()`

After the centipede drawing loop:

</> Python

```
for chain in self.chains:
    for index, segment in enumerate(chain):
        center = (segment.col * CELL + CELL // 2, segment.row * CELL + CELL // 2)
        pygame.draw.circle(screen, (240, 200, 60) if index == 0 else (80, 220, 90), center, CELL // 2)
```

add:

</> Python

```
for row, col, lifetime in self.sparks:
    center = (col * CELL + CELL // 2, row * CELL + CELL // 2)
    pygame.draw.circle(screen, (255, 255, 255), center, 6)
```

This gives us a small white spark that remains visible for about **0.2 seconds** after a segment is hit.

Now test Task 3

Run:

```
</> Bash
```

```
python3 game.py
```

Shoot a centipede segment.

You should see a **brief white spark at the segment's last position**.

Do not commit yet.

Reply **“done”** after testing.

done

Step 13: Commit Task 3 separately

Task 3 is now implemented and tested. The README requires **separate commits for each task**, so commit this work now.

 Lab_4_VibeCoding_Student_handout

Run:

```
</> Bash
```

```
git status
```

Then:

```
</> Bash
```

```
git add game.py
```

Then:

</> Bash

```
git commit -m "Add segment hit spark effect"
```

Finally:

</> Bash

```
git status
```

You should have a clean working tree.

Do not start Task 4 yet.

Reply **"done"** once the commit succeeds.

done

Step 14: Start Task 4

Now we move to the **final coding task**: `wave_speed_bonus(wave)` .

The README says it is used here:

</> Python

```
effective_tick = TICK / (wave_speed_bonus(self.wave) or 1)
```

So a multiplier greater than `1` makes later waves faster. The README specifically suggests:

 Python

```
1 + 0.15 * (wave - 1)
```

 README

For your LLM history, send me this prompt exactly:

"Task 3 is implemented, tested, and committed. Now implement Task 4 only. Please inspect `wave_speed_bonus(wave)` and how its return value is used to calculate `effective_tick`. The README suggests `1 + 0.15 * (wave - 1)`. Explain why this makes later waves faster and give the minimal implementation. Do not modify any other functionality."

Send that prompt next.

"Task 3 is implemented, tested, and committed. Now implement Task 4 only. Please inspect `wave_speed_bonus(wave)` and how its return value is used to calculate `effective_tick`. The README suggests `1 + 0.15 * (wave - 1)`. Explain why this makes later waves faster and give the minimal implementation. Do not modify any other functionality."

The proposed implementation is straightforward.

The game calculates:

 Python

```
effective_tick = TICK / (wave_speed_bonus(self.wave) or 1)
```

So if the multiplier increases, `effective_tick` becomes smaller. Since segments move whenever the timer reaches `effective_tick`, a smaller tick means they move **more frequently**, making later waves faster.  README

The README's suggested formula:

`</> Python`

```
1 + 0.15 * (wave - 1)
```

gives:

- Wave 1 → 1.00×
- Wave 2 → 1.15×
- Wave 3 → 1.30×
- Wave 4 → 1.45×

So wave 1 keeps the original speed, while every subsequent wave gets 15% more speed multiplier.

Exact change

Replace:

`</> Python`

```
def wave_speed_bonus(wave):  
    """Return an extra tick-rate multiplier for centipede segments at the given wave, or None for the default speed."""  
    pass
```

with:

`</> Python`

```
def wave_speed_bonus(wave):  
    """Return an extra tick-rate multiplier for centipede segments at the given wave, or None for the default speed."""
```

```
return 1 + 0.15 * (wave - 1)
```

This is the **only code change for Task 4**.

Make the change, save the file, and reply **"done"**.

done

Step 15: Test Task 4

Task 4 is implemented. Now test it before committing.

Run:

```
</> Bash
```

```
python3 game.py
```

You do **not** need to play through an entire wave manually if that takes too long. The important behavior is:

- **Wave 1:** normal speed
- **Wave 2:** faster
- **Wave 3:** faster again
- Later waves continue increasing in speed.

The formula you implemented is exactly the one suggested in the README.  [README](#)

If the game runs without errors and the wave speed increases, reply **"done"**. Then we will make the final Task 4 commit.

done

Step 16: Commit Task 4

Task 4 is tested successfully. Now create the **fourth and final task-specific commit**.

Run:

```
</> Bash  
git status
```

Then:

```
</> Bash  
git add game.py
```

Then:

```
</> Bash  
git commit -m "Increase speed for later waves"
```

Finally:

```
</> Bash  
git status
```

You should see a clean working tree.

Do not push yet. After this, we will do the final verification and record the required **10-second after-changes video**. The handout requires the post-change video and updated code as deliverables.  [Lab_4_VibeCoding_Student_handout](#)

Reply **"done"** after the commit succeeds.

done

Step 17: Final verification + after-changes video

All **4 task commits** are now complete.

Before pushing anything, do one final run of the game:

```
</> Bash
```

```
python3 game.py
```

Check these four things:

1. 🍄 **Mushroom:** disappears only after 4 hits.
2. 🍄 **Mushroom color:** changes according to its HP.
3. ✨ **Segment hit:** brief white spark appears when a segment is destroyed.
4. 🐛 **Waves:** later waves move faster.

These correspond to the README's expected behavior and four tasks.  [README](#)

Then record the final video 🎥

Record **about 10 seconds** showing the updated game and, preferably, demonstrate at least one of the new features plus the fixed mushroom behavior.

The handout requires a **10-second video after all changes**.  [Lab_4_VibeCoding_Student_handout](#)

Do not push yet.

Once you've recorded the final video, reply **"done"**.

done

Step 18: Final Git verification

The coding and testing are complete. Before pushing, verify the commit history.

Run:

```
</> Bash
```

```
git log --oneline -4
```

You should see **four separate commits**, corresponding to:

1. Fix mushroom durability
2. Implement mushroom color
3. Add segment hit spark effect
4. Increase speed for later waves

Then run:

```
</> Bash
```

```
git status
```

It should say the working tree is clean.

Do not push yet. Send me the output of `git log --oneline -4` and `git status` , and I will verify that your submission history is correct before we do the final push.

The verification is complete. Please now give me the final submission checklist based strictly on the README and lab handout, including exactly what files I need to upload under the Lab-4 folder and whether anything else is required. Also tell me how to export this complete chat history as a PDF/DOC for submission.

Yes. Based **strictly on the uploaded README and Lab 4 handout**, here is your final checklist.

What you must submit

The **handout** says your Lab-4 folder should contain:  Lab_4_VibeCoding_Student_handout

 Plain text

```
Lab-4/  
├─ before_video  
├─ after_video  
├─ game.py  
└─ chat_history.pdf (or .doc)
```

Specifically:

Item	Required?	What it should contain
 Before video		~10 seconds of the original game showing the mushroom bug
 After video		~10 seconds after your changes, showing the fixed game/features
 Updated code		Your final <code>game.py</code>

Item	Required?	What it should contain
 Chat history		Complete LLM/VibeCoding conversation exported as PDF/DOC

The README's submission checklist similarly requires the **before video, after video, and Chat/LLM page link with complete chat history**.



README

Git requirements

These aren't additional files, but they **are required assignment steps**:

- Separate commit for **each task**.
- Push your code to **your own repository**.
- **Do not raise a PR** to the main SETAPESU26 repository.  Lab_4_VibeCoding_Student_handout

You should therefore have four task commits:

 Plain text

```
Fix mushroom durability
Implement mushroom color
Add segment hit spark effect
Increase speed for later waves
```

Chat history PDF/DOC

Since the handout specifically says the chat history should be exported as a **doc/pdf**, the safest approach is:

ChatGPT → open this conversation → Share/Export → export/download the conversation, if your ChatGPT interface provides that option.

If it gives you a **PDF**, use that directly.

If it only gives you the conversation as a webpage/text, you can use your browser's:

Print → Save as PDF

Make sure the exported document contains the **entire conversation from the beginning**, not just the Task 4 portion. This conversation is particularly important because the lab asks you to use the LLM as your debugging/pair-programming partner.

Final Lab-4 folder

I'd organize it like:

</> Plain text

```
Lab-4/  
├─ before_video.mp4  
├─ after_video.mp4  
├─ game.py  
└─ chat_history.pdf
```

Nothing else is explicitly required by the handout. The handout does not specify any particular filenames for these files.

 Lab_4_VibeCoding_Student_handout

One distinction: the README mentions the **Chat/LLM page link**, while the handout says **chat history exported as a doc/pdf**. Since your instructor's handout explicitly asks for the exported chat history, I would include the PDF/DOC, and if your repo/submission form has a place for the ChatGPT page link, include that too. The sources don't explicitly say that the link must also be uploaded as a separate file.  README